Field Hours

CSCE 603 Database Systems and Applications

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Relations, schemas, and keys

You name a relation, list its attributes, and then store tuples that match that list. A tuple is one value for each attribute, and the relation is a set, so an exact duplicate tuple is not a second fact. Take a Student relation with 4 attributes: id, last, first, and year. The schema is the attribute list. The rows you insert later have to follow it, including the year stored as an integer rather than a phrase.

A key is a set of attributes that stays unique and that contains no smaller unique set. You choose one candidate key as the primary key. In a section of 30 students, last name fails as a key when 2 students share it, while student_id works if each id is issued once. When no single column identifies a row, you use a composite key of 2 attributes.

From a schema down to one key.

Relation name and attributes

Tuples, with duplicates collapsed

Candidate keys inside the attributes

One primary key you declare

Two Ada rows, one key

1. You insert (17, Ada, Lee) twice and (18, Ada, Ng) once. The duplicate (17, Ada, Lee) is one tuple, so 2 tuples remain.

2. student_id values 17 and 18 differ, so id separates the rows even though both first names are Ada.

3. The pair (first, last) is not a key. A later row (19, Ada, Lee) is legal, and the names collide while the ids do not.

Problem 1. The 2 rows you have today differ in last name. Is (first, last) a key?

Answer. No. A key has to stay unique in every legal state, not only in the rows you happen to store. A third student can share both names while holding a new student_id.

Problem 2. A section is identified by term and course_no, but two sections of one course meet in the same term. Is that pair a key?

Answer. No. Those two sections share term and course_no, so the pair is not unique. Add a section attribute. A composite key is required when no single column identifies the row, and this pair is still too small.

Lecture 2 · 90 minutes

Relationships become relations

You draw entities for things that need their own attributes, and you draw relationships for facts that connect them. Cardinality tells you how many entities on one side can match one entity on the other. A student and a department with a many-to-one advisor link means each student has 1 department, while a department has many students. You do not store a list inside a cell. You store that link as attributes that refer to keys.

A many-to-many relationship cannot hide inside either entity. You give it a new relation whose primary key is the pair of foreign keys, plus any attribute that belongs to the link itself. Enrollment of 80 students across 6 courses is that case: a grade is not a property of the student alone or of the course alone. It is a property of the pair, so the grade column lives on the enrollment relation.

From a link to a table.

Name the two entities -> Read the cardinality -> Make a relation for the link -> Put grade on that pair

Where a grade is allowed to live

1. Student and Course are entities. One student takes many courses, and one course enrolls many students.

2. Enrollment(student_id, course_id, grade) is the link. The key is the pair of ids, not either id alone.

3. A grade column on Student would give that student one cell. It cannot record grades for 4 courses.

Problem 1. Why do you refuse to store a list of course ids inside the student row?

Answer. A list inside a cell is not one value for an attribute. The link has to be attributes that refer to keys, so each course id can stand in its own enrollment row.

Problem 2. Each of 80 students enrolls in 2 of the 6 courses. How many enrollment rows do you get?

Answer. 160. Each enrollment is one pair, and 80 students times 2 courses is 160 rows. The 6 courses bound the catalog, not the row count.

Lecture 3 · 90 minutes

Selection, products, and joins

You read rows with SELECT, you keep columns with the select list, and you keep rows with WHERE. A join matches tuples by a predicate, usually equality of a foreign key and a primary key. If you write two relations in FROM and forget the predicate, you get the Cartesian product. On 20 students and 5 courses that product has 100 rows, almost all of them meaningless pairs you did not intend to report.

An inner join keeps a tuple only when the predicate finds a partner. You use it for enrollment rows that must point at a real student and a real course. A student with 0 enrollments disappears from that result, which is correct if you asked who is enrolled, and wrong if you asked for every student. Put the student on the preserved side of an outer join when absence is a fact you still need to see.

How an unbounded product shrinks.

1. FROM builds 20 times 5 rows

2. The predicate keeps real pairs

3. Inner join drops unmatched students

4. Outer join can keep them

A product you did not ask for

1. Student has 20 rows and Course has 5. FROM Student, Course with no predicate yields 100 rows.

2. Adding the enrollment equalities keeps only pairs that appear in Enrollment. The 100 was not that set.

3. A student with no enrollment row is missing from the inner join. A left outer join from Student still returns that student.

Problem 1. Why is 100 the wrong answer to 'which students take which courses' when you have 20 students and 5 courses?

Answer. 100 is the Cartesian product, 20 times 5. Most of those pairs were never enrolled. You need a join predicate through Enrollment, not the bare product.

Problem 2. A student has 0 enrollment rows. What does an inner join to Enrollment return for that student, and what does a left outer join from Student return?

Answer. The inner join drops the student, because no partner matches. The left outer join keeps the student and leaves the enrollment columns empty. Use it when absence itself is a fact you need.

Lecture 4 · 90 minutes

Groups, aggregates, and nested queries

WHERE filters rows before groups exist. GROUP BY then bundles the surviving rows, and the select list may contain only group attributes and aggregates such as COUNT, SUM, and AVG. HAVING filters groups after the aggregate is computed. If you want courses with average grade at least 80, the average does not exist at WHERE time. You put that test in HAVING, and you put student-level tests, such as grade above 0, in WHERE.

A nested query is a SELECT whose result you use inside another predicate. You use IN when the inner query returns a set of keys. Use a scalar subquery only when the inner query is guaranteed to return 1 row. The courses of one instructor can be that inner set, and the outer query finds students enrolled in it. If the inner query returns 2 rows, a scalar comparison fails instead of picking a row.

Filter, group, then filter groups.

WHERE keeps rows -> GROUP BY builds groups -> Aggregates compute a number -> HAVING keeps groups

Averages against a line at 80

1. Course A has grades 70 and 90, B has 100, and C has 50 and 60. With no WHERE, all 5 grades remain.

2. GROUP BY course computes averages 80, 100, and 55.

3. HAVING average >= 80 keeps A and B and drops C. A WHERE grade >= 80 would have deleted 70, 50, and 60 first, so C would vanish instead of reporting 55.

Problem 1. What is course A's average, and does HAVING average >= 80 keep it?

Answer. A's grades are 70 and 90, so the average is 80. The HAVING test keeps a group at 80, so A remains, together with B at 100. C at 55 does not.

Problem 2. You grouped by course_id. Why is student name illegal in the select list?

Answer. The select list may contain only the group attributes and aggregates. Student name is neither, so a course group does not have one student name to return.

Lecture 5 · 90 minutes

Dependencies and three anomalies

A functional dependency is a rule on legal tuples: if two tuples agree on a set of attributes X, they must agree on Y. You write that as X determines Y. Keys are the dependencies you already used for identity, but other dependencies hide in the attributes. In a table that stores employee_id, dept, and manager, each employee has 1 dept, and each dept has 1 manager, so employee_id determines dept and dept determines manager.

If you store the manager beside every employee, that fact is copied once per person. Change it in 1 of 3 rows and the other 2 keep the old name: an update anomaly. Delete the last employee and you also delete the only copy of the manager: a deletion anomaly. A new department with 0 employees has nowhere to store its manager: an insertion anomaly.

One fact, copied three times.

Stored rows

(1, Lab, Kim)

(2, Lab, Kim)

(3, Lab, Kim)

What the copy causes

Edit 1 row, 2 stay stale

Delete all 3, Lab is gone

No row for a new dept

Three copies of Kim

1. The rows are (1, Lab, Kim), (2, Lab, Kim), and (3, Lab, Kim). employee_id determines dept, and dept determines manager.

2. You change row 1 to Patel and leave rows 2 and 3 as Kim. Lab now shows two managers.

3. Delete all 3 employees and the fact that Lab has a manager disappears with them.

Problem 1. Which dependency is transitive in the employee table?

Answer. employee_id determines manager, because employee_id determines dept and dept determines manager. manager is not itself the key. That chain is the transitive dependency.

Problem 2. You update the manager in 1 of the 3 Lab rows. How many rows are stale?

Answer. 2. Those two rows still show the old manager, so the department no longer has one manager fact.

Lecture 6 · 90 minutes

Third normal form and a split

First normal form requires each cell to hold one value, not a list. Second normal form applies when the key has 2 or more attributes: a non-key attribute must not depend on only part of that key. Third normal form forbids a transitive dependency, where a non-key attribute determines another non-key attribute. You check the employee table against the third condition, because manager depends on dept and dept depends on employee_id.

Boyce-Codd normal form asks for something stricter than third normal form: every determinant must be a candidate key. You reach a practical design by decomposing along a dependency. Split the employee table into two relations, Emp(employee_id, dept) and Dept(dept, manager). The shared attribute dept lets you join them back. With 9 employees and 2 departments, the manager name is stored 2 times after the split, not 9.

Four checks, from cells to keys.

One value in each cell

No partial key dependency

No transitive dependency

Every determinant is a key

Pull the manager into its own row

1. EmpDept violates third normal form: dept determines manager, and dept is not a candidate key. employee_id is the key.

2. Replace it with Emp(employee_id, dept) and Dept(dept, manager). dept is the key of Dept, so the join on dept matches at most one manager.

3. Nine employees and 2 departments store each manager 2 times, not 9. Each employee still has one dept.

Problem 1. A cell stores two phone numbers. Does the relation satisfy first normal form?

Answer. No. First normal form requires one value in each cell, not a list. Split the phones into their own rows or their own relation.

Problem 2. The key is (student_id, course_id), and student_name depends only on student_id. Which normal form fails?

Answer. Second normal form. The key has 2 attributes, and student_name depends on only one of

them. Move student_name to a Student relation whose key is student_id.

Lecture 7 · 90 minutes

Constraints and views

A primary key constraint rejects a second row with the same key. A foreign key rejects a reference to a missing parent, and it can block a delete of a parent that still has 3 children. CHECK rejects a predicate that fails, such as a grade outside 0 to 100. NOT NULL rejects a missing value in a column you marked mandatory. You declare these on the schema so the bad write fails at once.

A view is a named query. You query it as if it were a relation, and the database merges the view definition into the outer query. A view of one base relation that only drops rows or columns can be updatable. A view that joins 2 relations or aggregates them is a report, and an INSERT through it is not a single base-row write. Use the view for the read you repeat, and use the base relations for the write.

A write meets the schema.

1. Check the primary key

2. Check each foreign key

3. Check CHECK and NOT NULL

4. Accept or reject the row

A grade that cannot be stored

1. CHECK requires grade between 0 and 100. An insert of 110 fails before any query can see it.

2. A foreign key requires course_id to exist. Citing course 999 fails when that course row is absent.

3. A view of students in year 4 can accept a name change. A view that averages grades by course cannot accept an insert, because that insert is not one base row.

Problem 1. A parent row still has 3 child rows. What does the foreign key do if you delete the parent?

Answer. It can block the delete while those 3 children remain. The lecture's rule is that a parent with 3 children is not removed out from under them.

Problem 2. Why do you refuse an INSERT into a view that joins 2 relations?

Answer. That view is a report. An insert through it is not a single base-row write, so you insert into the base relations instead.

Lecture 8 · 90 minutes

A schema you can defend

You can now turn a short description into entities, relations, keys, and dependencies, and you can see when a table should be split. You can write the SQL that joins those relations, groups them, and rejects a row that breaks a constraint. Given 1 description of students, courses, and grades, you should be able to place the grade on the enrollment relation, argue the key, and compute a per-course average without mixing WHERE and HAVING.

The design is finished only when the dependencies and the queries agree. If you decompose into 2 relations so that dept determines manager in its own relation, you must still join back on dept and get each employee's manager one time. A foreign key from employee.dept to department.dept keeps that join from inventing a department you never stored. Normalization without the constraint is a drawing. The constraint without a dependency to remove is only a refusal to insert.

One description, one schema.

Place grade on the pair -> Check each determinant -> Declare the foreign keys -> Group in HAVING, not WHERE

Enrollment that survives a join

1. Build Student(student_id, name), Course(course_id, title), and Enrollment(student_id, course_id, grade). The grade sits on the pair.

2. student_id determines name, course_id determines title, and the pair determines grade. Each determinant is a key, so the schema is in Boyce-Codd normal form.

3. Foreign keys point from Enrollment to both parents. An average by course uses GROUP BY and HAVING on AVG(grade), not WHERE.

Problem 1. You split managers into Dept(dept, manager). An employee row names dept Q, and Q is not in Dept. What happens, and which two earlier mechanisms are doing the work?

Answer. The foreign key rejects the insert. The split put each manager in Dept so it is stored once per department, and the foreign key stops a dept value that would not join to any manager. Without the constraint you could store Q and then fail to find a manager. Without the split the manager fact would still be copied onto every employee.

Problem 2. Course P has grades 80 and 100. Course Q has grade 70. You want averages of at least 85, and the grade must not live on Student. Which course remains, and why is Enrollment the relation you group?

Answer. P remains: (80 + 100) / 2 = 90, and 90 >= 85. Q's average is 70, so Q is dropped by HAVING. The grade is a property of the student-course pair, so it lives on Enrollment, which is the relation you group by course.